

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)**

Факультет **Прикладной информатики**

Образовательная программа **Мобильные и облачные технологии**

Направление подготовки **09.03.03 Прикладная информатика**

О Т Ч Е Т

по ЛАБОРАТОРНОЙ РАБОТЕ №6

Тема: Работа с БД в СУБД MongoDB

Выполнили: Сатрио Субагио K3240

Преподаватель : Говорова Марина Михайловна

Дата 27.05.2026

Санкт-Петербург 2026

1 Цель работы

овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

2 Выполнение работы

2.1 ЛАБОРАТОРНОЙ РАБОТЕ № 6.1

Методы справки и статистики

```
test> db.help

Database Class:

  getMongo           Returns the current database connection
  getName            Returns the name of the DB
  getCollectionNames Returns an array containing the names of all collections in the current d
atabase.
  getCollectionInfos Returns an array of documents with collection information, i.e. collectio
n name and options, for the current database.
  runCommand         Runs an arbitrary command on the database.
  adminCommand       Runs an arbitrary command against the admin database.
  aggregate           Runs a specified admin/diagnostic pipeline which does not require an unde
rlying collection.
  getSiblingDB       Returns another database without modifying the db variable in the shell e
nvironment.
  getCollection       Returns a collection or a view object that is functionally equivalent to
using the db.<collectionName>.
  dropDatabase       Removes the current database, deleting the associated data files.
  createUser         Creates a new user for the database on which the method is run. db.create
User() returns a duplicate user error if the user already exists on the database.
  updateUser         Updates the user's profile on the database on which you run the method. A
n update to a field completely replaces the previous field's values. This includes updates to the user's roles array.
  changeUserPassword Updates a user's password. Run the method in the database where the user
is defined, i.e. the database you created the user.
  logout             Ends the current authentication session. This function has no effect if t
he current session is not authenticated.
  dropUser           Removes the user from the current database.
```

```
test> db.help()

Database Class:

  getMongo           Returns the current database connection
  getName            Returns the name of the DB
  getCollectionNames Returns an array containing the names of all collections in the current d
atabase.
  getCollectionInfos Returns an array of documents with collection information, i.e. collectio
n name and options, for the current database.
  runCommand         Runs an arbitrary command on the database.
  adminCommand       Runs an arbitrary command against the admin database.
  aggregate           Runs a specified admin/diagnostic pipeline which does not require an unde
rlying collection.
  getSiblingDB       Returns another database without modifying the db variable in the shell e
nvironment.
  getCollection       Returns a collection or a view object that is functionally equivalent to
using the db.<collectionName>.
  dropDatabase       Removes the current database, deleting the associated data files.
  createUser         Creates a new user for the database on which the method is run. db.create
User() returns a duplicate user error if the user already exists on the database.
  updateUser         Updates the user's profile on the database on which you run the method. A
n update to a field completely replaces the previous field's values. This includes updates to the user's roles array.
  changeUserPassword Updates a user's password. Run the method in the database where the user
is defined, i.e. the database you created the user.
  logout             Ends the current authentication session. This function has no effect if t
he current session is not authenticated.
  dropUser           Removes the user from the current database.
```

```
test> db.stats()
{
  db: 'test',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}
test> |
```

Создание БД learn И Создание коллекции и вставка документа

```
test> use learn
switched to db learn
learn> show dbs
admin    40.00 KiB
config   12.00 KiB
local    64.00 KiB
learn> db.unicorns.insertOne({ name: 'Aurora', gender: 'f', weight: 450 })
{
  acknowledged: true,
  insertedId: ObjectId('6a0f2931dcc6280936abc114')
}
learn> |
```

Список текущих коллекций

```
learn> show collections
unicorns
learn> |
```

Переименование коллекции

```
learn> db.unicorns.renameCollection("dragon")
{ ok: 1 }
learn> |
```

Статистика коллекции

```
learn> db.dragon.stats()
{
  ok: 1,
  capped: false,
  wiredTiger: {
    metadata: { formatVersion: 1 },
    creationString: 'access_pattern_hint=none,allocation_size=4KB,app_metadata=(formatVersion=1),assert=(commit_timestamp=none,durable_timestamp=none,read_timestamp=none,write_timestamp=off),block_allocation=best,block_compressor=snappy,block_manager=default,cache_resident=false,checksum=on,colgroups=,collator=,columns=,dictionary=0,disaggregated=(page_log=,storage_tier=none),encryption=(keyid=,name=),exclusive=false,extractor=,format=btree,huffman_key=,huffman_value=,ignore_in_memory_cache_size=false,immutable=false,import=(compare_timestamp=oldest_timestamp,enabled=false,file_metadata=,metadata_file=,panic_corrupt=true,repair=false),in_memory=false,ingest=,internal_item_max=0,internal_key_max=0,internal_key_truncate=true,internal_page_max=4KB,key_format=q,key_gap=10,leaf_item_max=0,leaf_key_max=0,leaf_page_max=32KB,leaf_value_max=64MB,log=(enabled=true),lsm=(auto_throttle=,bloom=,bloom_bit_count=,bloom_config=,bloom_hash_count=,bloom_oldest=,chunk_count_limit=,chunk_max=,chunk_size=,merge_custom=(prefix=,start_generation=,suffix=),merge_max=,merge_min=),memory_page_image_max=0,memory_page_max=10M,os_cache_dirty_max=0,os_cache_max=0,prefix_compression=false,prefix_compression_min=4,source=,split_deepen_min_child=0,split_deepen_per_child=0,split_pct=90,stable=,tiered_storage=(auth_token=,bucket=,bucket_prefix=,cache_directory=,local_retention=300,name=,object_target_size=0),type=file,value_format=u,verbose=[],write_timestamp_usage=none',
    type: 'file',
    uri: 'statistics:table:collection-24be0188-8850-4c8d-be62-7519fc90eebf',
    version: 'WiredTiger 12.0.0: (November 15, 2024)',
    autocommit: {
```

Удаление коллекции

```
learn> db.dragon.drop()
true
learn> |
```

Удаление базы данных

```
learn> db.dropDatabase()
{ ok: 1, dropped: 'learn' }
learn> |
```

2.2 ЛАБОРАТОРНОЙ РАБОТЕ № 6.2

CRUD -операции

```
learn> db.unicorns.insertMany([
  {name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63},
  {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43},
  {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182},
  {name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99},
  {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80},
  {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40},
  {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39},
  {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2},
  {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33},
  {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54},
  {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'}
]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a0f2c1ddcc6280936abc115'),
    '1': ObjectId('6a0f2c1ddcc6280936abc116'),
    '2': ObjectId('6a0f2c1ddcc6280936abc117'),
    '3': ObjectId('6a0f2c1ddcc6280936abc118'),
    '4': ObjectId('6a0f2c1ddcc6280936abc119'),
    '5': ObjectId('6a0f2c1ddcc6280936abc11a'),
    '6': ObjectId('6a0f2c1ddcc6280936abc11b'),
    '7': ObjectId('6a0f2c1ddcc6280936abc11c'),
    '8': ObjectId('6a0f2c1ddcc6280936abc11d'),
    '9': ObjectId('6a0f2c1ddcc6280936abc11e'),
    '10': ObjectId('6a0f2c1ddcc6280936abc11f')
  }
}
learn> |
learn> var document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
learn> db.unicorns.insertOne(document);
{
  acknowledged: true,
  insertedId: ObjectId('6a0f2c97dcc6280936abc120')
}
learn> |
```

Выборка и сортировка

```
learn> db.unicorn.find({gender: 'm'}).sort({name: 1})

learn> db.unicorn.find({gender: 'f'}).sort({name: 1}).limit(3)
```

Проекция данных

```
learn> db.unicorns.find().sort({$natural: -1})
```

```
[
  {
    _id: ObjectId('6a0f2c97dcc6280936abc120'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a0f2c1ddcc6280936abc11f'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a0f2c1ddcc6280936abc11e'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
]
```

```
learn> db.unicorns.find(
```

```
|   { gender: 'm' },
|   { name: 1, weight: 1, _id: 0 }
| )
```

```
[
  { name: 'Horny', weight: 600 },
  { name: 'Unicrom', weight: 984 },
  { name: 'Roooooodles', weight: 575 },
  { name: 'Kenny', weight: 690 },
  { name: 'Raleigh', weight: 421 },
  { name: 'Pilot', weight: 650 },
  { name: 'Dunx', weight: 704 }
]
```

```
learn> |
```

```
learn> db.unicorns.find({gender:'f', loves:'carrot'}).limit(1)
```

```
[
  {
    _id: ObjectId('6a0f2c1ddcc6280936abc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

```
learn> |
```

```

learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a0f2c1ddcc6280936abc116'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> |
learn> db.unicorns.find({}, {name: 1, loves: {$slice: 1}, _id: 0})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
learn> |

```

Вложенные объекты (Коллекция towns)

```

learn> db.towns.insertMany([
  | {name: "Punxsutawney", population: 6200, last_sensus: ISODate("2008-01-31"), famous_for: [""], mayor: {name: "Jim Wehrle"},
  | },
  | {name: "New York", population: 22200000, last_sensus: ISODate("2009-07-31"), famous_for: ["status of liberty", "food"],
  |   mayor: {name: "Michael Bloomberg", party: "I"}},
  | {name: "Portland", population: 528000, last_sensus: ISODate("2009-07-20"), famous_for: ["beer", "food"], mayor: {name: "Sam Adams", party: "D"}}
  | ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a1552585e00c32948abc114'),
    '1': ObjectId('6a1552585e00c32948abc115'),
    '2': ObjectId('6a1552585e00c32948abc116')
  }
}
learn> |

```

```

learn> db.towns.find({"mayor.party": "I"}, {name: 1, mayor: 1, _id: 0})
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
learn> |

```

Курсоры и JavaScript

```
learn> var cursor = db.unicorns.find(getMales).sort({name: 1}).limit(2);

learn> cursor.forEach(function(obj) { print(obj.name); })
MongoInvalidArgumentError: Query filter must be a plain object or ObjectId
learn> var cursor = db.unicorns.find({ gender: 'm' }).sort({ name: 1 }).limit(2);

learn> cursor.forEach(function(obj) {
|   print(obj.name);
| });
Dunx
Horny

learn> |
```

Агрегация (Group By)

```
learn> db.unicorns.aggregate([
|   {$group: {_id: "$gender", count: {$sum: 1}}}
| ])
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]
learn> |
```

InsertOne (вместо save)

```
learn> db.unicorns.insertOne({name:'Barney', loves:['grape'], weight: 340, gender:'m'})
{
  acknowledged: true,
  insertedId: ObjectId('6a1554be5e00c32948abc117')
}
learn> |
```

UpdateMany и UpdateOne (\$set)

```
learn> db.unicorns.updateMany(
|   { gender: 'm' },
|   { $inc: { vampires: 5 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
learn> db.unicorns.find({gender: 'm'}, {name: 1, vampires: 1, _id: 0})
[
  { name: 'Horny', vampires: 68 },
  { name: 'Unicrom', vampires: 187 },
  { name: 'Rooooooodles', vampires: 104 },
  { name: 'Kenny', vampires: 44 },
  { name: 'Raleigh', vampires: 7 },
  { name: 'Pilot', vampires: 59 },
  { name: 'Dunx', vampires: 170 },
  { name: 'Barney', vampires: 5 }
]
learn> |
```

```
learn> db.unicorns.updateMany(
|   { gender: 'm' },
|   { $inc: { vampires: 5 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
learn> |
```

```
learn> db.towns.updateOne(
|   { name: 'Portland' },
|   { $unset: { "mayor.party": "" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find({name: 'Portland'})
[
  {
    _id: ObjectId('6a1552585e00c32948abc116'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
learn> |
```

Increment (\$inc)

```
learn> db.unicorns.updateMany(  
|   { gender: 'm' },  
|   { $inc: { vampires: 5 } }  
| )  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 8,  
  modifiedCount: 8,  
  upsertedCount: 0  
}  
learn> |
```

AddToSet with \$each

```
learn> db.unicorns.updateOne(  
|   { name: 'Pilot' },  
|   { $addToSet: { loves: 'chocolate' } }  
| )  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}  
learn> db.unicorns.find({name: 'Pilot'})  
[  
  {  
    _id: ObjectId('6a0f2c1ddcc6280936abc11e'),  
    name: 'Pilot',  
    loves: [ 'apple', 'watermelon', 'chocolate' ],  
    weight: 650,  
    gender: 'm',  
    vampires: 59  
  }  
]  
learn> |
```

```

learn> db.unicorns.updateOne(
|   { name: 'Aurora' },
|   { $addToSet: { loves: { $each: ['sugar', 'lemon'] } } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a0f2c1ddcc6280936abc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> |

```

DeleteMany

```

learn> db.towns.deleteMany({ "mayor.party": "I" })
{ acknowledged: true, deletedCount: 0 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a1552585e00c32948abc114'),
    name: 'Punxsutawney',
    population: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ '' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('6a1552585e00c32948abc116'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
learn> |

```

```
learn> db.towns.deleteMany({ "mayor.party": "I" })
{ acknowledged: true, deletedCount: 0 }
learn> |
```

```
learn> db.towns.deleteMany({"mayor.party": "I"})
| db.towns.find()
[
  {
    _id: ObjectId('6a1552585e00c32948abc114'),
    name: 'Punxsutawney',
    population: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ '' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('6a1552585e00c32948abc116'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
learn> |
```

```
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 2 }
learn> show collections
towns
unicorns
learn> |
```

Ссылки (Links)

```
learn> var horny = db.unicorns.findOne({name: 'Horny'});
learn> print("Habitat ID: " + horny.habitat_id);
Habitat ID: forest
learn> var habitat = db.habitats.findOne({_id: horny.habitat_id});
learn> print("Habitat Name: " + habitat.name);
Habitat Name: Enchanted Forest
```

Unique Index

```
learn> db.unicorns.createIndex({ name: 1 }, { unique: true })
name_1
learn> |
```

```
learn> for(let i = 0; i < 100000; i++) {
|   db.numbers.insertOne({ value: i });
| }
{
  acknowledged: true,
  insertedId: ObjectId('6a15b46c29d889ae41ad47b4')
}
```

```
learn> db.numbers.createIndex({ value: 1 })
value_1
learn> db.numbers.find({ value: 99999 }).explain("executionStats")
{
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 105,
    totalKeysExamined: 0,
    totalDocsExamined: 100000,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: { value: { '$eq': 99999 } },
      nReturned: 1,
      executionTimeMillisEstimate: 62,
      works: 100001,
      advanced: 1,
      needTime: 99999,
      needYield: 0,
      saveState: 4,
      restoreState: 4,
      isEOF: 1,
      nss: 'learn.numbers',
      direction: 'forward',
    },
  },
}
```

```
executionStats: {
  executionSuccess: true,
  nReturned: 1,
  executionTimeMillis: 34,
  totalKeysExamined: 1,
  totalDocsExamined: 1,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 1,
    executionTimeMillisEstimate: 24,
    works: 2,
    advanced: 1,
    needTime: 0,
    needYield: 0,
    saveState: 1,
    restoreState: 1,
    isEOF: 1,
    nss: 'learn.numbers',
    docsExamined: 1,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 1,
      executionTimeMillisEstimate: 24,
      works: 2,
      advanced: 1,
      needTime: 0,
      needYield: 0,
```

ВЫВОДЫ

В ходе выполнения лабораторных работ № 6.1 и № 6.2 были успешно освоены практические навыки работы с документо-ориентированной СУБД MongoDB.

По результатам Лабораторной работы № 6.1 («Введение в СУБД MongoDB. Установка MongoDB»):

Изучены архитектурные особенности MongoDB, включая формат хранения данных BSON, концепцию коллекций и документов, а также отличия от реляционных СУБД (отсутствие жесткой схемы, внешних ключей и SQL-запросов).

Выполнена установка и настройка сервера MongoDB (mongod) и клиентской консоли (mongosh).

Освоены базовые команды администрирования: создание и переключение между базами данных (use), просмотр списка БД и коллекций (show dbs, show collections), получение статистики (db.stats(), collection.stats()).

Получен опыт управления жизненным циклом объектов БД: неявное создание коллекций при вставке данных, их переименование (`renameCollection`) и полное удаление (`drop`, `dropDatabase`).

По результатам Лабораторной работы № 6.2 («Работа с БД в СУБД MongoDB»):

CRUD-операции: Глубоко изучены методы вставки (`insertOne`, `insertMany`), выборки (`find`, `findOne`), обновления (`updateOne`, `updateMany`) и удаления (`deleteOne`, `deleteMany`) документов. Освоено использование операторов модификации: `$set` (изменение полей), `$inc` (инкремент чисел), `$push` и `$addToSet` (работа с массивами), `$unset` (удаление полей).

Сложные запросы: Научился применять логические операторы (`$gt`, `$lt`, `$or`, `$and`, `$in`) и операторы существования (`$exists`) для фильтрации данных. Освоены техники проекции (выбор конкретных полей) и сортировки результатов (`sort`, `limit`, `skip`).

Работа со структурами данных: Получены навыки работы с вложенными объектами (через точечную нотацию, например, `major.party`) и массивами (использование `$slice` для выборки элементов массива).

Агрегация и курсоры: Изучены методы агрегации данных (`count`, `distinct`, `aggregate` с `$group`) для статистического анализа. Освоено программирование на JavaScript внутри консоли MongoDB для обработки курсоров (итерация через `forEach`).

Связи и индексы: Реализованы ручные ссылки между коллекциями (нормализованная модель данных) на примере коллекций `unicorns` и `habitats`. Изучена роль индексов в оптимизации производительности: создан уникальный индекс, проанализирован план выполнения запроса с помощью `explain("executionStats")`, что наглядно продемонстрировало преимущество индексного поиска (IXSCAN) перед полным сканированием коллекции (COLLSCAN) на больших объемах данных.

